

Apostila Prática: Go (Golang)

Guia Prático e Conceitual — Módulo: Pacotes Importantes

• os, io & bufio | • net/http & context | • encoding/json & text/template

Parte 1: I/O, Sistema de Arquivos, Ciclo de Vida e JSON

01 Manipulação de arquivos 16:56

O pacote `os` lida com criação, leitura e escrita no sistema de arquivos. Para arquivos grandes, usa-se `bufio.NewReader` ou `bufio.NewScanner` para leitura em buffers ou linha por linha, evitando estourar a memória.

```
f, err := os.Create("arquivo.txt")
if err != nil { panic(err) }
defer f.Close()

f.WriteString("Gravando dados no disco")
```

02 Realizando chamada HTTP 04:50

O método `http.Get` do pacote `net/http` executa requisições GET simples. O corpo da resposta (`resp.Body`) é um stream do tipo `io.ReadCloser` que deve ser fechado após o uso e lido com `io.ReadAll`.

```
resp, err := http.Get("https://api.github.com")
if err != nil { panic(err) }
defer resp.Body.Close()

body, _ := io.ReadAll(resp.Body)
```

03 Defer 04:43

A diretiva `defer` adia a execução de uma função até o momento exato em que a função circundante retorna. Executa em ordem **LIFO** (Last In, First Out) e é o padrão oficial para liberar recursos (fechar conexões, arquivos e destravar mutexes).

```
file, _ := os.Open("dados.csv")
defer file.Close() // Garante fechamento mesmo se ocorrer erro/pânico
```

04 Trabalhando com json 16:14

O pacote `encoding/json` converte structs para JSON (`Marshal`) e JSON para structs (`Unmarshal`). Mapeia campos usando *struct tags* (ex: ``json:"nome_campo"``).

```
type Usuario struct {
    Nome string `json:"nome"`
    Email string `json:"email,omitempty"`
}

bytes, _ := json.Marshal(Usuario{Nome: "Ana"}) // Serialização
var u Usuario
_ = json.Unmarshal(bytes, &u)                 // Desserialização
```

05 Busca CEP 13:48

Aplicação prática unindo HTTP, JSON e manipulação de arquivos: consome a API do ViaCEP com `http.Get`, decodifica a resposta diretamente em uma struct com `json.NewDecoder(resp.Body).Decode(&dados)` e persiste o resultado formatado em disco.

```
type ViaCEP struct {
    Cep      string `json:"cep"`
    Logradouro string `json:"logradouro"`
    Bairro   string `json:"bairro"`
    Localidade string `json:"localidade"`
    Uf       string `json:"uf"`
}

// 1. Requisição HTTP
resp, err := http.Get("https://viacep.com.br/ws/01001000/json/")
if err != nil { panic(err) }
defer resp.Body.Close()

// 2. Decode direto da Stream de rede
var dados ViaCEP
if err := json.NewDecoder(resp.Body).Decode(&dados); err != nil {
    panic(err)
}

// 3. Salva formatado em arquivo de texto
f, err := os.Create("cep.txt")
if err != nil { panic(err) }
defer f.Close()

fmt.Fprintf(f, "CEP: %s, Rua: %s, Cidade: %s-%s", dados.Cep, dados.Logradouro, dados.Localidade, dados.Uf)
```

Parte 2: Servidores Web, Handlers e Roteamento

06 Iniciando com HTTP 08:57

Para inicializar um servidor web nativo em Go, associamos padrões de rota a funções *handler* via `http.HandleFunc` e ouvimos a porta com `http.ListenAndServe`.

```
http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
    w.Write([]byte("Servidor HTTP em Go ativo!"))
})
http.ListenAndServe(":8080", nil)
```

07 Manipulando Headers 06:50

Cabeçalhos HTTP definem tipos de conteúdo (Content-Type) e status de retorno. **Regra crucial:** cabeçalhos e status devem ser configurados antes de escrever qualquer byte no `ResponseWriter`.

```
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusOK)
w.Write([]byte(`{"status":"ok"}`))
```

08 Criando função para BuscaCEP 05:32

Isolamento da regra de negócio em um handler HTTP reutilizável: captura o CEP via parâmetros da URL (`r.URL.Query().Get("cep")`), valida o formato e orquestra a consulta externa.

09 Finalizando resposta para o servidor HTTP 07:41

Uso de `json.NewEncoder(w).Encode(dados)` para serializar e enviar os dados diretamente ao fluxo de saída de rede sem precisar alocar um slice intermediário de bytes na memória.

10 ServeMux 11:38

O `DefaultServeMux` é global e pode ser inseguro em bibliotecas de terceiros. A boa prática é instanciar roteadores multiplexadores dedicados com `http.NewServeMux()` para controle total de injeção de dependência.

```
mux := http.NewServeMux()
mux.HandleFunc("/api/v1/health", HealthCheckHandler)
http.ListenAndServe(":8080", mux)
```

11 FileServer 05:04

Serve arquivos estáticos (HTML, CSS, imagens, JS) a partir de um diretório físico no disco usando `http.FileServer` combinado com `http.Dir` e `http.StripPrefix`.

```
fs := http.FileServer(http.Dir("./public"))
http.Handle("/static/", http.StripPrefix("/static/", fs))
```

Parte 3: Renderização com Templates (HTML / Text)

12 Iniciando com templates 05:51

Os pacotes `text/template` e `html/template` (com sanitização contra ataques XSS) permitem intercalar dados dinâmicos de structs em strings ou páginas web usando as tags `{{ .Campo }}`.

```
tpl := template.New("curso")
tpl, _ = tpl.Parse("Bem-vindo ao curso de {{ .Nome }}!")
tpl.Execute(os.Stdout, struct{ Nome string }{"Go"})
```

13 Template.Must 03:00

Função auxiliar utilitária que encapsula a compilação do template e dispara um `panic` imediato caso ocorra erro sintático. Usado exclusivamente na inicialização da aplicação para falhar cedo (**fail fast**).

```
var t = template.Must(template.ParseFiles("template.html"))
```

14 Templates com arquivo externo 06:51

Separação limpa de responsabilidades entre código Go e layouts, carregando templates externos a partir de arquivos `.html` com `template.ParseFiles("home.html")`.

15 Templates com webserver 03:57

Envio da renderização do template diretamente para a resposta HTTP passando o `http.ResponseWriter` como o `io.Writer` de destino no método `t.Execute(w, dados)`.

16 Compondo templates 06:09

Criação de layouts base reutilizáveis (cabeçalho, navegação e rodapé) através de `template.ParseGlob("*.html")` e das diretivas internas `{{ define "conteudo" }}` e `{{ template "conteudo" . }}`.

17 Mapeando funções nos templates 06:39

Extensão de recursos dentro dos templates com injeção de funções Go personalizadas (como conversão de texto, formatação de moeda ou datas) usando `template.FuncMap`.

```
funcs := template.FuncMap{
    "maiuscula": strings.ToUpper,
}
tpl := template.New("view").Funcs(funcs)
```

18 Documentação para Template

Referência essencial: Consulte a documentação oficial do pacote `html/template` para dominar controle de fluxo condicional (`{{ if }}`), iteração em listas (`{{ range }}`) e context-aware escaping.

Parte 4: Clientes HTTP Avançados e Contextos

19 HttpClient com Timeout 05:31

O cliente padrão `http.DefaultClient` **não possui timeout configurado**, podendo prender goroutines indefinidamente sob falhas de rede. É mandatória a criação de instâncias de `http.Client` com timeout explícito em produção.

```
client := &http.Client{
    Timeout: 5 * time.Second,
}
resp, err := client.Get("https://servico-externo.com")
```

20 Trabalhando com Post 07:02

Envio de payloads e corpos de requisição com o verbo POST usando `http.Post` ou serializando structs para `bytes.NewBuffer(jsonData)` com o tipo MIME `application/json`.

```
payload := bytes.NewBuffer([]byte(`{"nome":"Item 1"}`))
resp, err := http.Post("https://api.com/itens", "application/json", payload)
```

21 Customizando Request 04:38

Para métodos além de GET/POST (como PUT, DELETE, PATCH) ou para injetar cabeçalhos de autenticação (`Authorization: Bearer <token>`), cria-se um objeto sob demanda com `http.NewRequest` e dispara-se com `client.Do(req)`.

```
req, _ := http.NewRequest(http.MethodPut, "https://api.com/recurso", body)
req.Header.Set("Authorization", "Bearer token-secreto")
resp, err := client.Do(req)
```

22 Trabalhando com HTTP usando Contextos 11:22

O pacote `context` propaga prazos (*deadlines*), cancelamentos e valores através das fronteiras de chamadas de API e goroutines. Com `http.NewRequestWithContext`, se a requisição estourar o tempo limite ou for abortada pelo cliente, os recursos de rede e banco são encerrados imediatamente.

```
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()

req, err := http.NewRequestWithContext(ctx, "GET", "https://api-lenta.com", nil)
resp, err := http.DefaultClient.Do(req)
```